

awk command

- awk is a powerful text-processing command in Linux used to analyze, filter, and manipulate structured data such as logs, CSV files, and command output.
- It works by scanning input line by line and performing actions based on patterns and fields.
- Processes text column-wise using fields (e.g., \$1, \$2)
- Ideal for parsing logs, reports, and delimited files
- Supports conditions, loops, and built-in functions
- Commonly used in shell scripts and data processing pipelines

syntax

```
awk [options] 'pattern {action}' input-file
```

```
# [options]: Controls AWK behavior (e.g., -F to set field separator)
# pattern: Specifies which lines to process (condition or regex)
# {action}: Defines what to do with matching lines (usually print)
```

options

- -F : to specify field separator
- -f : to specify script file
- -v : to define a variable

Built-In Variables In Awk

- \$0, \$1, \$2, ...
 - \$0 - represents entire input line
 - \$1, \$2, ... - represent individual fields (columns).
- NR (Number of Records)
 - Keeps track of the current line number being processed.
- NF (Number of Fields)
 - Stores the total number of fields in the current line.
- FS (Field Separator)
 - Defines the character used to split fields in the input.
 - Default is whitespace
- RS (Record Separator)
 - Specifies how input records are separated.
 - Default is a newline character

- OFS (Output Field Separator)
 - Controls the separator used between fields in output.
 - Default is a space and is applied when using commas in print.
- ORS (Output Record Separator)
 - Defines how output lines are separated.
 - Default is a newline, and it is automatically added after each print.

```
# print all lines
awk '{print}' students.csv

# print 1st and 2nd column of each line
awk -F ',' '{print $1, $2}' students.csv

# print msg before processing
awk -v msg="Student Details:" 'BEGIN {print msg}{print}' students.csv

# print students whose roll no is less than 20
awk -v limit=20 '$1 < limit {print $1, $2 }' students.csv

# search lines with a keyword
awk '/pune/ {print}' students.csv

# print all input lines with line numbers
awk '{print NR, $0}' students.csv

# print first and last column of input line
awk -F ',' '{print $1,$NF}' employee.txt

# display lines from 2 to 3
awk 'NR == 2 , NR == 3 {print NR, $0}' students.csv

# to print empty line if any
awk 'NF <= 0 {print NR}' students.csv

# to print number of lines in input file
awk 'END { print NR }' students.csv

# find length of longest line in a file
awk '{ if (length($0) > max) max = length($0) } END { print max }' students.csv
```

Types of commands

- Internal commands
 - commands are part of shell program only.
 - No seperate executable is present
 - eg. alias, unalias, cd

- External commands
 - commands are not part of shell program
 - Seperate executable is available in any one of the bin directory
 - eg. ls, cp, mv

Types of Command execution

- Synchronous execution
 - Shell waits for command to be completed.
 - Default execution
- Asynchronous execution
 - Shell doesn't wait for command to be completed.
 - To execute a command asynchronously, use "&" at the end of the command.
 - terminal> gedit &

Shell Scripting/Programming

- Shell Script - set of commands with some programming constructs, written for shell
- commands - shell commands
- Programming construct - control flow structures, functions, arrays
 - Decision control structure (if else)
 - Selection control structure (case)
 - Loop control structure (while, until, for)
- Shell scripts are interpreted by shell

Applications

- Shell scripts are used by administration people for installations, setups, automations
- Administrative tasks e.g. User creations, Troubleshooting, Router/Firewall config on commandline, etc.
- Automation of tasks e.g. Cron jobs, Scripts for building softwares, etc.
- Application installers

Limitations

- Slow execution -- Interpreted. Cannot be used for performance centric applications.

Procedure to write and execute scripts

```
* Step 1 - create file with extension .sh
* $ vim script.sh
* Step 2 - execute the script
* Option 1
* $ bash script.sh

* Option 2
* $ chmod +x script.sh
* $ ./script.sh
```